Bio Bloom Software Design Specification (SDS) v2.0

Reconciled with the codebase

Item	Detail
Project	Bio Bloom: Artisanal Food E-Commerce Platform
Course	SENG 34213
Department	Department of Software Engineering, Faculty of Science, University of Kelaniya
Document	Software Design Specification (SDS)
Version	2.0 (supersedes SDS v01 dated 2026/04/07)
Date	25 October 2026
Status	Final, reconciled with the code
Team	Ovindu Pathirana (SE/2021/059), Prageeth Ekanayake (SE/2021/049): backend, database, cron, security, DevOps. Chamindu Dilshan (SE/2021/058), Sanduni Ayeshika (SE/2021/039): frontend. I.G.R. Ravindu Laksara (SE/2021/028): mobile
Reference code	Monorepo `biobloom` (`apps/*`, `packages/*`, `infra/*`, `docs/*`)

Version history

Version	Date	Author	Summary
01	2026/04/07	Project team	Initial SDS draft
2.0	25 October 2026	Project team	Design reconciled with the implemented code. Every statement is traced to a repository path

---

Table of contents

1. Introduction
2. Changes from SDS v01
3. System architecture
4. Module catalogue
5. Database architecture
6. Core sequence diagrams
7. Security architecture
8. Scheduled jobs
9. API surface, error handling and internationalisation
10. Deployment and CI/CD
11. ADR index

12. Known gaps and verification register

1. Introduction

1.1 Purpose

This document describes the design of Bio Bloom as it is built. SDS v01 was written before the code. Since then the team made design decisions, recorded as ADR-001 to ADR-014 in `docs/adr/`. This version brings the design and the code back into line.

Readers are the project supervisor, the examiners and future maintainers.

1.2 Scope

Bio Bloom sells packaged artisanal food (jams, chilli sauces, gift boxes). The design covers:

- FIFO batch inventory with expiry tracking (food safety).
- Subscribe & Save recurring orders with automated daily billing.
- Weight-based shipping fees.
- Product variants and bundles.
- A multi-seller marketplace (ADR-002), with a seller portal.
- Order fulfilment and delivery through a Flutter rider app.

1.3 Conventions

- Paths such as `apps/api/src/inventory/inventory.service.ts` point to the evidence. A line number is added where it helps (`file.ts:101`).
- "Not implemented" means no code exists for it. "Not measured" means nobody ran a measurement.
- All times are UTC (ADR-008).
- Diagrams are Mermaid blocks.

1.4 Sources

Source	Path
SDS v01 (30 pages)	`Bio Bloom SDS Final (1).pdf`
Prisma schema	`packages/prisma/prisma/schema.prisma`
Migrations	`packages/prisma/prisma/migrations/*/migration.sql`
API source	`apps/api/src/*`
Web apps	`apps/storefront`, `apps/admin`, `apps/seller`
Mobile app	`apps/delivery`
Decisions	`docs/adr/ADR-001` to `ADR-014`
API reference	`docs/api-reference.md`
Pipelines and infrastructure	`.github/workflows/`, `infra/terraform/`, `docker-compose.yml`

2. Changes from SDS v01

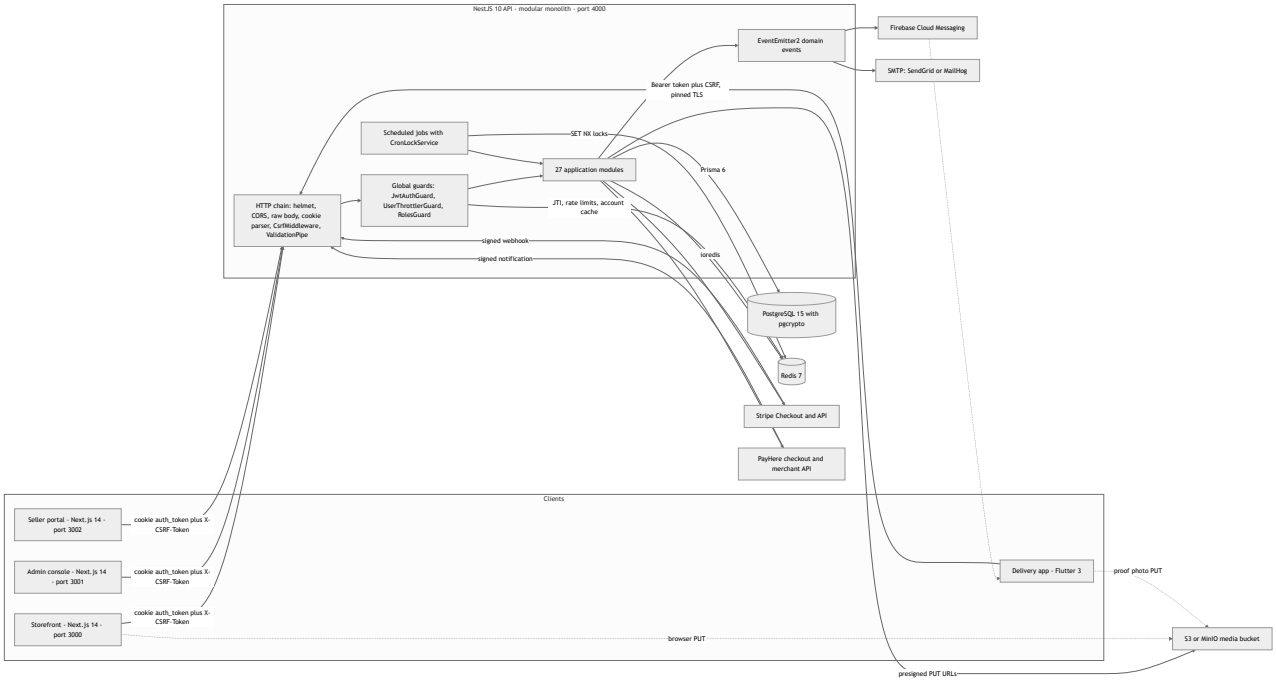
#	Area	SDS v01	Implemented design (v2.0)	Evidence
1	Business model	Single vendor (§1.2)	Multi-seller marketplace. Sellers apply, an admin approves. Every product and bundle belongs to one seller.	ADR-002, `apps/api/src/sellers/`
2	Client applications	Storefront, admin dashboard, delivery app	Four clients: storefront, admin console, **seller portal**, Flutter delivery app.	`apps/storefront`, `apps/admin`, `apps/seller`, `apps/delivery`
3	Data model	14 primary entities	**25 Prisma models (25 tables)**. Added: ProductImage, Seller, Payout, PromoCode, RefundRequest, FavoriteProduct, AuditLog, DeliveryUser, DeliveryAssignment, SupportTicket, CartItem.	`packages/prisma/prisma/schema.prisma`
4	Roles and actor tables	3 roles, 2 tables (Customer, AdminUser)	5 roles (`CUSTOMER`, `ADMIN`, `FULFILMENT_STAFF`, `SELLER`, `DELIVERY`). 4 separate actor tables: `customer`, `admin_user` (ADMIN or FULFILMENT_STAFF), `seller`, `delivery_user`.	ADR-005, `schema.prisma` enum `Role`, `apps/api/src/auth/types/auth.types.ts`
5	FIFO allocation	`checkStock` picks one batch with `ORDER BY expiry_date ASC LIMIT 1`, then `decrementBatch`	`InventoryService.decrementForVariant` locks **all** valid batches of the variant with `SELECT ... FOR UPDATE`, ordered by `expiry_date ASC, batch_number ASC`, and draws across several batches. One order item is written per batch drawn.	ADR-007, `apps/api/src/inventory/inventory.service.ts:89–143`
6	When stock is decremented	On payment confirmation (§5.3)	At order creation, in the same transaction that creates the `PENDING_PAYMENT` order. Stock is released on failed payment, customer cancel, or the 30-minute stale-order job.	ADR-011, `apps/api/src/orders/orders.service.ts`, `orders/stale-order-cron.service.ts`
7	Billing cron	`@Cron('0 6 * * *')` inside SubscriptionService	Separate `BillingCronService` with a per-day Redis lock and a unique `billing_cycle_key` per subscription cycle. Declined charges release stock and leave a FAILED payment row.	`apps/api/src/subscriptions/billing-cron.service.ts`
8	Scheduled jobs	Billing and expiry sweep. Cron runs under a dedicated service account (§7.5)	Three jobs (billing, expiry sweep, stale orders). They run inside the API process, not under a separate service account.	`infra/terraform/README.md` ("Deviation: cron jobs run inside the API service")

#	Area	SDS v01	Implemented design (v2.0)	Evidence
9	Webhook verification	HMAC-SHA256 for all gateways (§4.2)	Stripe: `Stripe.webhooks.constructEvent` (HMAC-SHA256 with a 300 s timestamp tolerance). PayHere: **MD5** `md5sig`, compared with `timingSafeEqual`. PayHere's protocol uses MD5, not HMAC-SHA256.	`payments/gateways/stripe.gateway.ts:34–49`, `payments/gateways/payhere.gateway.ts:64–87`
10	Payment modes	Sandbox or live credentials	`PAYMENTS_MODE=mock` or `live`. Mock simulates the hosted page through the same webhook code path. Mock is refused when `NODE_ENV=production`.	ADR-012, `apps/api/src/payments/payments.config.ts`
11	Notifications	SendGrid API via EventEmitter2	EventEmitter2 events, 20 event types. Email through nodemailer SMTP (SendGrid SMTP in the deployed design, MailHog locally). FCM push to riders on `delivery.assigned`.	ADR-003, `apps/api/src/notifications/`
12	Redis usage	JWT blocklist, shipping rate cache	JTI blocklist, rate-limit counters, cron locks, account-standing cache (60 s), shipping-rate cache (300 s), payment-setup sessions (1 h). See §3.4.	`apps/api/src/redis/redis.service.ts`, `app.module.ts`
13	Token encryption key	AES-256 key managed by AWS KMS	`pgp_sym_encrypt` / `pgp_sym_decrypt` with the key from `SUBSCRIPTION_TOKEN_ENC_KEY`, bound as a SQL parameter. No `cipher-algo` option is passed, so pgcrypto's default cipher is used (AES-128), not AES-256. In the Terraform design the key is an SSM SecureString encrypted with a KMS key.	`apps/api/src/subscriptions/token-cipher.service.ts:31–50`, `infra/terraform/ssm.tf`
14	Batch creation endpoint	`POST /api/admin/products/:id/batches`	`POST /api/admin/products/:productId/variants/:variantId/batches` (batches belong to variants).	`apps/api/src/batches/batches.controller.ts`
15	New features	Not in SDS v01	Promo codes, partial refunds, review moderation, support tickets, delivery retries (ADR-013), presigned uploads (ADR-014), Sinhala storefront (ADR-010).	`apps/api/src/*`, `docs/adr/`
16	Staging deployment	AWS ECS Fargate, RDS, ElastiCache, ALB (§8)	Staging runs on **Render** (API, PostgreSQL 15, Key Value) and **Vercel** (storefront), deployed by GitHub Actions. `infra/terraform` holds the AWS ECS design, which is the documented production target.	§10, `infra/terraform/README.md`
17	Monorepo	Not specified	One pnpm workspace. The Flutter app is in the repo but outside the workspace.	ADR-001, `pnpm-workspace.yaml`

3. System architecture

3.1 System topology

Bio Bloom is a **modular monolith**. One NestJS 10 API serves four clients over HTTPS JSON. The API owns all business rules and is the only component that talks to the database.



3.2 Technology stack (as built)

Layer	Technology	Version in code	Evidence
Web frontends	Next.js App Router, Tailwind CSS	`next` 14.2.5 (monorepo)	`apps/*/package.json`
Storefront i18n	next-intl	`en`, `si`	`apps/storefront/i18n/routing.ts`
Mobile	Flutter 3, dio, flutter_secure_storage, firebase_messaging	see pubspec	`apps/delivery/pubspec.yaml`
Backend	NestJS on Node 20	`@nestjs/core` ^10.4.0, `.nvmrc` = 20	`apps/api/package.json`, `.nvmrc`
ORM	Prisma	^6.0.0	`packages/prisma/package.json`
Database	PostgreSQL with pgcrypto	15	`docker-compose.yml`, ADR-004
Cache and locks	Redis via ioredis	7 (`redis:7-alpine`)	`docker-compose.yml`
Rate limiting	`@nestjs/throttler` with Redis storage	^6.2.1, `@nest-lab/throttler-storage-redis`	`apps/api/src/app.module.ts`

Layer	Technology	Version in code	Evidence
Scheduler	`@nestjs/schedule`	^4.1.1	`apps/api/package.json`
Payments	Stripe SDK, PayHere HTTP API	`stripe` ^17.7.0	`apps/api/src/payments/gateways/`
Email	nodemailer (SMTP)	^6.9.15	`apps/api/src/notifications/mailer.service.ts`
Push	firebase-admin	^13.10.0	`apps/api/src/notifications/push.service.ts`
Object storage	AWS SDK v3 S3 (MinIO locally)	^3.1155.0	`apps/api/src/uploads/uploads.service.ts`
Package manager	pnpm	10.33.0	`package.json` `packageManager`

3.3 Layered architecture

The API keeps the four tiers from SDS v01, implemented with NestJS building blocks.

Tier	Responsibility	Implementation
Presentation	UI, routing, rendering, calling the API	Next.js 14 apps (Server Components for catalogue pages, Client Components for cart and checkout). Flutter app for riders.
Application (HTTP)	Transport concerns: headers, auth, validation, routing	`config/http-app.ts` (helmet, CORS, raw-body capture, cookie parser, `ValidationPipe` returning 422), `common/middleware/csrf.middleware.ts`, global guards in `app.module.ts`, 30 controllers.
Business logic	Business rules per domain	One service per module, for example `InventoryService`, `OrdersService`, `OrderStatusService`, `BillingCronService`, `ShippingService`, `PaymentsService`, `NotificationsService`. Services call each other through Nest dependency injection.
Data persistence	Storage, transactions, locking, caching	`PrismaService` (Prisma 6 over PostgreSQL 15), `$transaction` blocks for multi-row work, `SELECT ... FOR UPDATE` for stock, `RedisService` for keys with a TTL, S3 for media.

Rules that keep the layers clean:

- Controllers hold no business logic. They read the DTO and the current user, then call a service.
- Only `InventoryService` changes batch quantities (`inventory.service.ts` header comment: "sole inventory access point").
- Only `OrderStatusService.transition` changes `order.status`. It checks the lawful transition table and writes an audit row (`apps/api/src/orders/order-status.service.ts`).
- Side effects that may fail (email, push) are sent after the commit through EventEmitter2, so they never block or roll back a request.

3.4 Redis usage (verified in code)

Use	Key pattern	TTL	Evidence
JWT revocation list	`jti:<jti>`	Remaining token lifetime (minimum 1 s)	`apps/api/src/redis/redis.service.ts:44–53`
Account-standing cache	`account:<actorType>:<id>`	60 s (`ACCOUNT_CACHE_TTL_SEC`)	`redis.service.ts`, `auth/strategies/jwt.strategy.ts`
Rate-limit counters	Managed by `ThrottlerStorageRedisService`	Throttler window (60 s or 15 min)	`apps/api/src/app.module.ts`, `common/guards/throttler.config.ts`
Cron job locks	`cron:<job>:<yyyy-mm-dd>`	3600 s (daily jobs), 240 s (stale orders)	`apps/api/src/common/locks/cron-lock.service.ts`
Shipping-rate cache	`shipping:rates`	300 s, deleted when an admin edits rates	`apps/api/src/shipping/shipping.service.ts`
Saved-card setup sessions	`payments:setup:<sessionId>`	3600 s	`apps/api/src/payments/payments.service.ts:540–558`

If Redis is down at start-up, `RedisService.onModuleInit` throws and the API does not start. If the lock cannot be taken, a cron job is skipped ("fail closed").

3.5 External services

Service	Purpose	Integration point
Stripe	International cards, hosted Checkout, saved cards (off-session charges)	`payments/gateways/stripe.gateway.ts`
PayHere	LKR payments, hosted form, tokenised recurring charges	`payments/gateways/payhere.gateway.ts`
SendGrid / SMTP	Transactional email	`notifications/mailer.service.ts`
Firebase (FCM)	Push to riders on new assignments	`notifications/push.service.ts`, `apps/delivery/lib/app/push_service.dart`
S3 (MinIO locally)	Product images and proof-of-delivery photos	`uploads/uploads.service.ts`

---

4. Module catalogue

The API registers these modules in `apps/api/src/app.module.ts`. The health controller is registered directly on `AppModule`.

Module	Responsibility	Key files (under `apps/api/src/`)
`prisma`	Database client and connection lifecycle	`prisma/prisma.service.ts`

Module	Responsibility	Key files (under `apps/api/src/`)
`redis`	ioredis wrapper: JTI list, account cache, raw client for other modules	`redis/redis.service.ts`
`common`	CSRF middleware, user-aware throttler guard, cron lock service, i18n exception filter, raw-body and `/api/v1` alias middleware	`common/middleware/csrf.middleware.ts`, `common/guards/user-throttler.guard.ts`, `common/locks/cron-lock.service.ts`
`config`	Environment validation (fail fast in production), HTTP app setup	`config/env.validation.ts`, `config/http-app.ts`
`i18n`	English and Sinhala message catalogue for the API	`i18n/i18n.service.ts`, `i18n/translations/en.json`, `si.json`
`audit`	Append-only audit log of privileged actions	`audit/audit.service.ts`
`auth`	Registration, multi-actor login, logout, `/me`, CSRF token, JWT strategy, guards, decorators	`auth/auth.service.ts`, `auth/auth.controller.ts`, `auth/strategies/jwt.strategy.ts`, `auth/guards/*`
`addresses`	Customer shipping addresses	`addresses/addresses.service.ts`
`sellers`	Seller application, admin approval and suspension, seller portal summary and orders	`sellers/seller.service.ts`, `sellers/seller-admin.controller.ts`, `sellers/seller-portal.service.ts`
`products`	Public catalogue, admin and seller product, variant and image management with ownership checks	`products/products.service.ts`, `products/products-admin.service.ts`
`bundles`	Gift-box bundles of variants	`bundles/bundles.service.ts`
`batches`	Batch creation and listing, daily expiry sweep	`batches/batches.service.ts`, `batches/expiry-cron.service.ts`
`inventory`	FIFO allocation under row locks, stock checks, restock	`inventory/inventory.service.ts`, `inventory/inventory.errors.ts`
`shipping`	Weight-bracket fee calculation with Redis cache, admin rate management	`shipping/shipping.service.ts`, `shipping/shipping-rates-admin.service.ts`
`cart`	Persisted cart with stock validation	`cart/cart.service.ts`
`promo-codes`	Promo evaluation and atomic redemption	`promo-codes/promo-codes.service.ts`
`payments`	Hosted checkout, webhooks, saved cards, off-session charges, refunds, mock mode	`payments/payments.service.ts`, `payments/gateways/*`, `payments/payments.config.ts`
`orders`	Quote, checkout, order views, cancellation, status state machine, stale-order job	`orders/orders.service.ts`, `orders/order-status.service.ts`, `orders/stale-order-cron.service.ts`
`subscriptions`	Subscribe & Save lifecycle, token encryption, daily billing	`subscriptions/subscriptions.service.ts`, `subscriptions/billing-cron.service.ts`, `subscriptions/token-cipher.service.ts`

Module	Responsibility	Key files (under `apps/api/src/`)
`fulfilment`	Staff work queue: PAID to PACKING to SHIPPED to DELIVERED	`fulfilment/fulfilment.service.ts`
`delivery`	Rider assignment, status updates, proof of delivery, retries, rider accounts	`delivery/delivery.service.ts`, `delivery/riders.service.ts`
`refunds`	Refund requests and admin approval (full or partial, optional restock)	`refunds/refunds.service.ts`
`reviews`	Product reviews with admin moderation	`reviews/reviews.service.ts`
`support`	Customer support tickets	`support/support.service.ts`
`analytics`	Admin dashboard KPIs, revenue series, top sellers	`analytics/analytics.service.ts`
`staff`	Admin and fulfilment-staff accounts	`staff/staff.service.ts`
`uploads`	Presigned S3 PUT URLs, magic-byte verification	`uploads/uploads.service.ts`
`notifications`	Domain events to email and push	`notifications/notifications.service.ts`, `mailer.service.ts`, `push.service.ts`, `templates.config.ts`
`health`	Liveness and readiness probes	`health/health.controller.ts`

Shared packages:

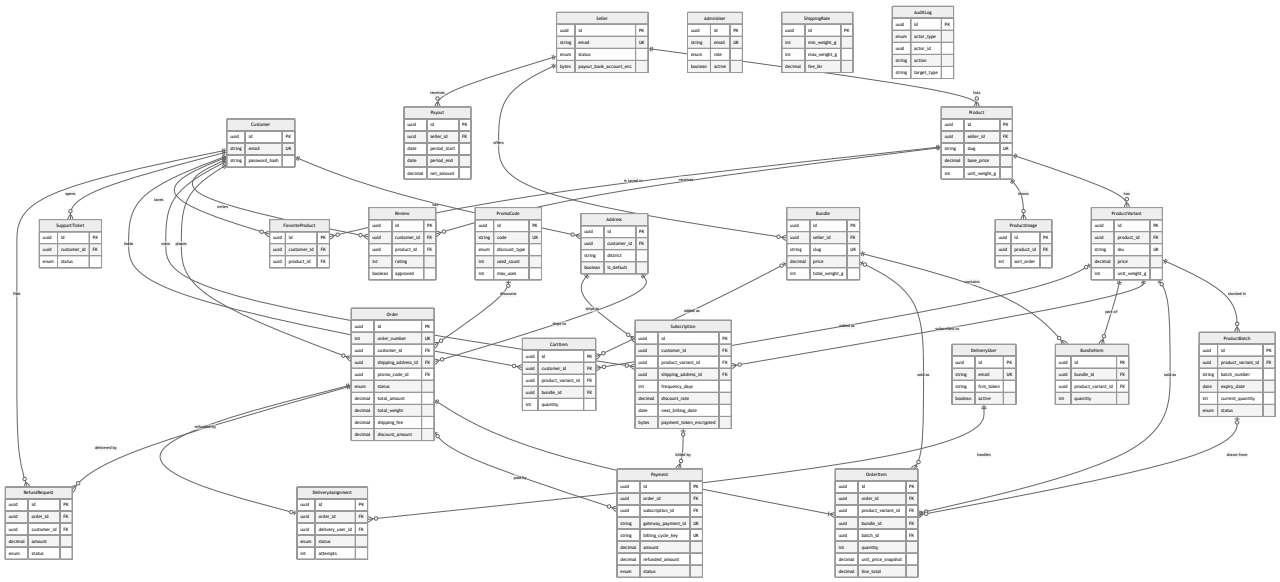
Package	Content
`packages/prisma`	Schema, migrations, seed, generated client
`packages/shared-types`	Enums and DTO types shared by the API and the web apps
`packages/ui`	Tailwind preset and shared React components

---

5. Database architecture

5.1 Entity-relationship diagram

All 25 models. Only the key columns are shown. Every table uses a UUID primary key.



`AdminUser`, `ShippingRate` and `AuditLog` have no foreign keys. `AuditLog.actor_id` can point to any of the four actor tables, so it has no foreign key on purpose (the FK was dropped in migration `20261022090000_spec_completion`).

5.2 Table catalogue

All primary keys are `UUID` (`@id @default(uuid()) @db.Uuid`). Columns are `snake_case` through `@map`. Money is `Decimal(12,2)`. Rates are `Decimal(5,4)`.

#	Model (table)	Purpose	Key foreign keys (on delete)	Implemented in API
1	Customer (`customer`)	Buyer account	none	Yes
2	AdminUser (`admin_user`)	Admin and fulfilment staff accounts, role `ADMIN` or `FULFILMENT_STAFF`	none	Yes
3	Seller (`seller`)	Marketplace seller with approval status	none (`approved_by_admin_id` is a plain column)	Yes
4	DeliveryUser (`delivery_user`)	Rider account with FCM token	none	Yes
5	Address (`address`)	Customer shipping address	`customer_id` (Cascade)	Yes
6	Product (`product`)	Sellable product with base weight, ingredients, allergens	`seller_id` (Restrict)	Yes
7	ProductImage (`product_image`)	Ordered image gallery	`product_id` (Cascade)	Yes
8	ProductVariant (`product_variant`)	Size or flavour with own SKU, price and unit weight	`product_id` (Cascade)	Yes
9	ProductBatch (`product_batch`)	Food-safety batch: dates, quantities, status	`product_variant_id` (Cascade)	Yes

#	Model (table)	Purpose	Key foreign keys (on delete)	Implemented in API
10	Bundle (`bundle`)	Gift box with fixed price and total weight	`seller_id` (Restrict)	Yes
11	BundleItem (`bundle_item`)	Variant and quantity inside a bundle	`bundle_id` (Cascade), `product_variant_id` (Restrict)	Yes
12	CartItem (`cart_item`)	Persisted cart line, variant or bundle	`customer_id`, `product_variant_id`, `bundle_id` (all Cascade)	Yes
13	Order (`order`)	Purchase with price, weight and fee snapshots	`customer_id` (Restrict), `shipping_address_id` (Restrict), `promo_code_id` (SetNull)	Yes
14	OrderItem (`order_item`)	One line per batch drawn	`order_id` (Cascade), `product_variant_id`, `bundle_id`, `batch_id` (Restrict, all nullable)	Yes
15	Payment (`payment`)	Gateway reference, amount, status, refunds, cycle key	`order_id` (SetNull), `subscription_id` (SetNull)	Yes
16	Subscription (`subscription`)	Subscribe & Save plan with encrypted payment token	`customer_id`, `product_variant_id`, `shipping_address_id` (all Restrict)	Yes
17	ShippingRate (`shipping_rate`)	Weight bracket and fee in LKR	none	Yes
18	PromoCode (`promo_code`)	Percentage or fixed discount with usage limits	none	Yes
19	RefundRequest (`refund_request`)	Customer refund request and admin decision	`order_id` (Restrict), `customer_id` (Restrict)	Yes
20	Review (`review`)	Rating and comment, shown after approval	`customer_id` (Cascade), `product_id` (Cascade)	Yes
21	SupportTicket (`support_ticket`)	Customer support request	`customer_id` (Cascade)	Yes
22	DeliveryAssignment (`delivery_assignment`)	One rider assignment per order, with attempts and proof	`order_id` (Cascade, unique), `delivery_user_id` (Restrict)	Yes
23	AuditLog (`audit_log`)	Append-only log of privileged actions	none (polymorphic `actor_id`)	Yes
24	Payout (`payout`)	Seller payout per period with commission	`seller_id` (Restrict)	**No** (schema only)
25	FavoriteProduct (`favorite_product`)	Customer wish list	`customer_id` (Cascade), `product_id` (Cascade)	**No** (schema only)

`Payout` and `FavoriteProduct` exist in the schema, but no API module, service or UI uses them (a search of `apps/api/src`, `apps/*/app` and `seed.ts` finds no reference).

5.3 Normalisation (3NF)

The schema is in Third Normal Form, apart from the documented snapshots in §5.4.

- **1NF.** Every column holds one value. Repeating groups are separate tables: variants, images, batches, bundle items, order items, cart items.
- **2NF.** Every table has a single-column UUID key, so no column can depend on part of a key. Join tables (`bundle_item`, `favorite_product`, `cart_item`) also use a surrogate key, plus a unique constraint on the natural pair.
- **3NF.** No non-key column depends on another non-key column. Examples:
  - A batch stores only its variant id. Product name, price and seller come through the relation.
  - An order stores `shipping_address_id`, not a copy of the address.
  - The seller of a product is found through `product.seller_id`, not stored on order items.
- **Actor separation (ADR-005).** Four identity tables instead of one `user` table with a role column. Each has its own lifecycle columns (`seller.status`, `admin_user.active`, `delivery_user.fcm_token`). A customer row can never gain a privileged role.

5.4 Intentional snapshots on Order and OrderItem

These columns copy values at checkout time. They break strict 3NF on purpose, so that history does not change when prices, rates or promo codes change later.

Table	Column	Type	What it freezes	Why
`order_item`	`unit_price_snapshot`	Decimal(12,2)	Variant price at checkout (or the bundle price share)	A later price change must not alter past orders or refunds
`order_item`	`line_total`	Decimal(12,2)	`unit_price × quantity` (discounted unit price for renewals)	Stable line amounts for refunds and seller revenue
`order_item`	`batch_id`	UUID	The batch the units came from	Food-safety traceability and restock to the right batch
`order`	`total_weight`	Decimal(12,2)	Total cart weight in grams	Logistics reporting and disputes
`order`	`shipping_fee`	Decimal(12,2)	Fee of the matching `shipping_rate` bracket	Rates can be edited by admins later
`order`	`discount_amount`	Decimal(12,2)	Promo or subscription discount	Promo codes can be changed or deactivated
`order`	`total_amount`	Decimal(12,2)	`items − discount + shipping`	The amount the gateway must charge (checked on webhook)
`order`	`currency`	Char(3), `LKR`	Currency of the charge	Reconciliation
`subscription`	`discount_rate`	Decimal(5,4)	Rate agreed at sign-up (`0.1000` today)	Honour the rate the customer agreed to

Notes:

- There is **no per-item weight snapshot**. Only the order total weight is frozen.

- There is **no product or variant name snapshot**. Order detail views join to the current variant and product (`orderDetailInclude` in `apps/api/src/orders/orders.service.ts`), so a renamed product shows its new name on old orders.
- Bundle lines are split into component lines. The bundle price is shared across components in proportion to list price, and rounding lands on the last component (`OrdersService.priceLines`).

5.5 Indexes and unique constraints

From `@@index`, `@@unique` and `@unique` in `schema.prisma`:

Table	Unique	Indexes	Main query served
`customer`	`email`	`email`	Login lookup
`admin_user`	`email`	`email`	Login lookup
`seller`	`email`	`status`	Pending applications list
`delivery_user`	`email`	none	Login lookup
`product`	`slug`	`seller_id`; `(category, active)`	Catalogue filter, seller's products
`product_image`	none	`(product_id, sort_order)`	Ordered gallery
`product_variant`	`sku`	`product_id`	Variants of a product
`product_batch`	`(product_variant_id, batch_number)`	`(product_variant_id, expiry_date)`; `(status, expiry_date)`	FIFO lock query; expiry sweep
`bundle`	`slug`	`seller_id`	Bundle page
`bundle_item`	`(bundle_id, product_variant_id)`	none	One line per variant in a bundle
`order`	`order_number` (serial, shown as `BB-000123`)	`(customer_id, placed_at)`; `status`	Order history; admin and stale-order queries
`order_item`	none	`order_id`	Order detail, restock
`subscription`	none	`(status, next_billing_date)`; `customer_id`	Daily billing selection
`shipping_rate`	`(min_weight_g, max_weight_g)`	none	Bracket set
`review`	`(customer_id, product_id)`	`(product_id, approved)`	One review per customer per product; approved list
`payment`	`gateway_payment_id`; `billing_cycle_key`	`order_id`; `subscription_id`	Webhook lookup; one charge per billing cycle

Table	Unique	Indexes	Main query served
`payout`	`(seller_id, period_start, period_end)`	`status`	(not used by code)
`promo_code`	`code`	`(code, active)`	Promo lookup
`refund_request`	none	`status`; `order_id`	Admin queue
`favorite_product`	`(customer_id, product_id)`	none	(not used by code)
`audit_log`	none	`(actor_type, actor_id)`; `(target_type, target_id)`; `created_at`	Audit search
`delivery_assignment`	`order_id`	`(delivery_user_id, status)`	Rider feed; one assignment per order
`support_ticket`	none	`(customer_id, status)`	Customer tickets
`cart_item`	`(customer_id, product_variant_id)`; `(customer_id, bundle_id)`	`customer_id`	One line per item per cart

5.6 Check constraints and extensions (migration SQL)

Prisma cannot declare CHECK constraints, so they are written in `packages/prisma/prisma/migrations/20261022090000_spec_completion/migration.sql` (lines 59-67):

Constraint	Rule
`product_batch_quantities_check`	`current_quantity >= 0 AND initial_quantity > 0`
`review_rating_check`	`rating BETWEEN 1 AND 5`
`cart_item_quantity_check`	`quantity > 0`
`order_item_quantity_check`	`quantity > 0`
`payment_amounts_check`	`amount >= 0 AND refunded_amount >= 0 AND refunded_amount <= amount`
`promo_code_usage_check`	`used_count >= 0 AND (max_uses IS NULL OR used_count <= max_uses)`

The initial migration (`20260516033651_initial/migration.sql:6`) runs `CREATE EXTENSION IF NOT EXISTS "pgcrypto"`. Docker Compose also runs `infra/postgres/init/01-pgcrypto.sql` (ADR-004).

Not enforced by a constraint (enforced only in code):

- `order_item` and `cart_item` need a variant or a bundle. No CHECK requires one of them.
- `payment` should belong to an order or a subscription. No CHECK requires one of them.
- `shipping_rate` brackets must not overlap. Only the exact `(min, max)` pair is unique.

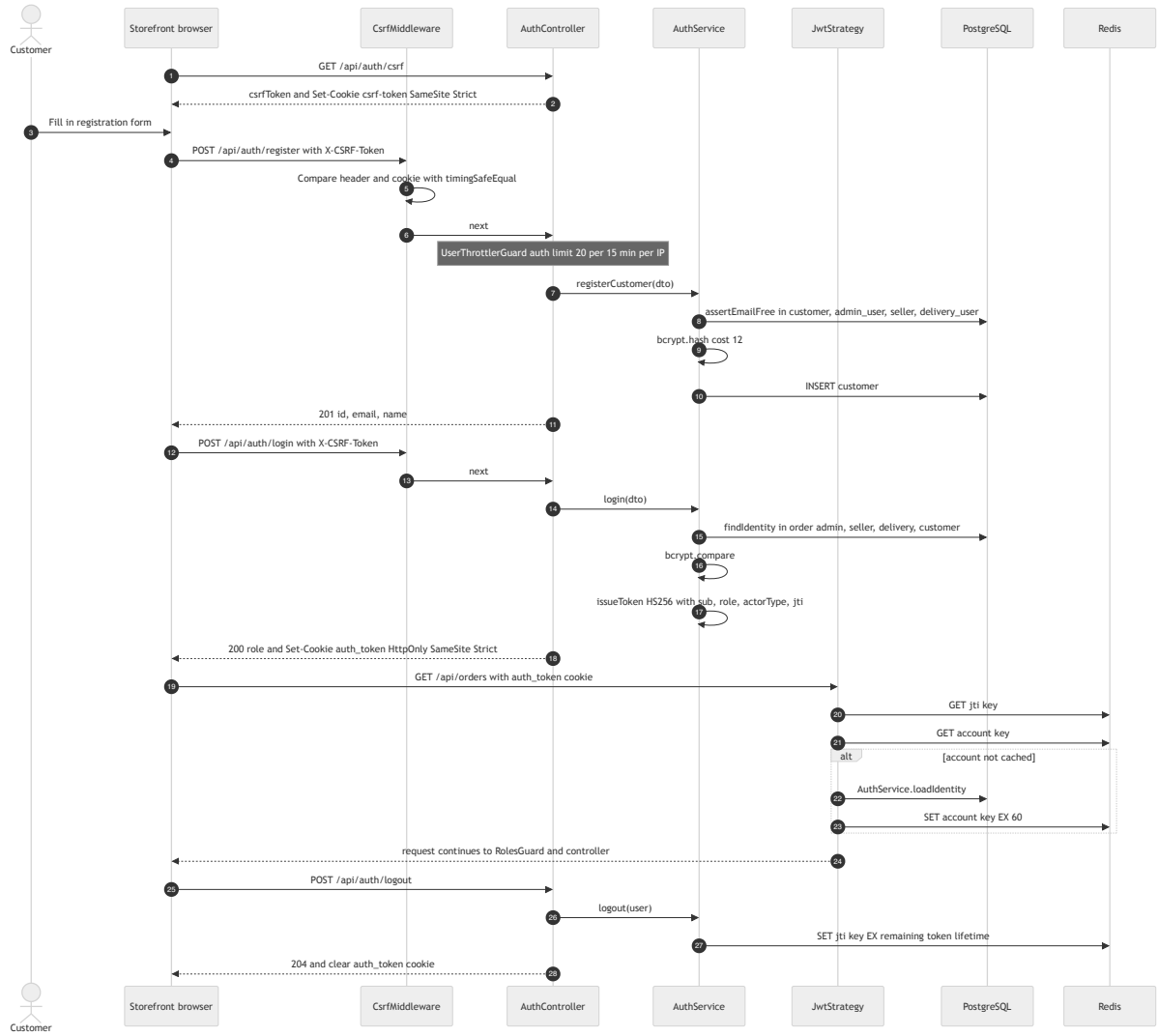
6. Core sequence diagrams

6.1 Customer registration and multi-actor JWT login

Walkthrough

1. The browser calls `GET /api/auth/csrf` (`AuthController.csrf`). The API sets the `csrf-token` cookie (`SameSite=Strict`, readable by JavaScript) and returns the same value.
2. `POST /api/auth/register` passes `CsrfMiddleware` (header must equal cookie, constant-time compare). The `auth` throttler allows 20 requests per 15 minutes per IP.
3. `AuthService.registerCustomer` lower-cases the email and calls `assertEmailFree`, which checks all four actor tables. It hashes the password with bcrypt cost 12 (`BCRYPT_COST`) and creates the `customer` row. Only customers can self-register.
4. `POST /api/auth/login` calls `AuthService.login`. `findIdentity` looks the email up in the order admin, seller, delivery, customer. A disabled admin or rider, or a seller who is not `APPROVED`, gets 403. An unknown email still runs a bcrypt compare, so timing does not reveal which emails exist.
5. `issueToken` signs an HS256 JWT with `sub`, `role`, `actorType` and a random `jti`. Customers get 15 days, all other actors 8 hours (`JWT_CUSTOMER_TTL`, `JWT_STAFF_TTL`).
6. The controller sets the `auth_token` cookie: `HttpOnly`, `SameSite=Strict`, `Secure` in production.
7. On every protected request, `JwtStrategy.validate` checks the signature, the Redis JTI list, then the account standing (cached in Redis for 60 s). A role change or deactivation gives 401.
8. `POST /api/auth/logout` writes `jti:<id>` to Redis with the token's remaining lifetime and clears the cookie.

The Flutter app uses the same login. It reads the token from the `Set-Cookie` header, stores it in secure storage, sends `Authorization: Bearer`, and also performs the CSRF double submit (`apps/delivery/lib/app/api_client.dart:65-82`).



6.2 FIFO stock allocation with row-level locking

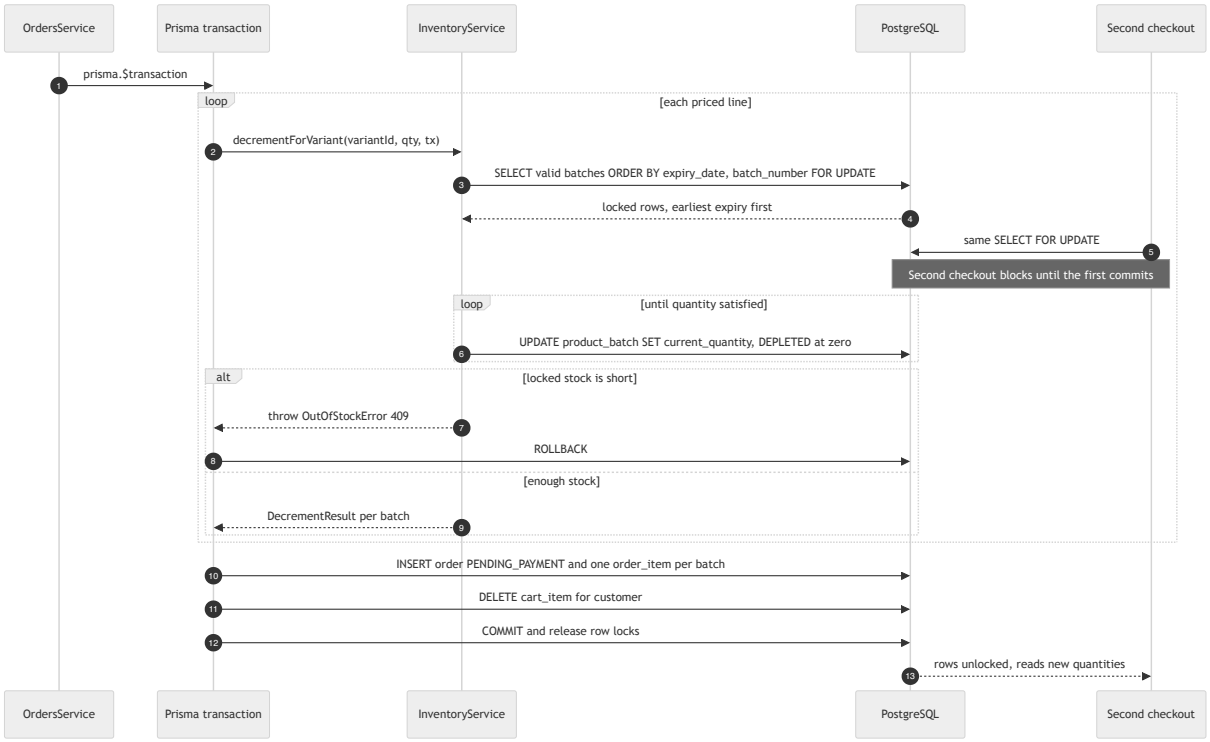
Walkthrough

1. `OrdersService.checkout` opens `prisma.$transaction`. For each priced line it calls `InventoryService.decrementForVariant(variantId, qty, tx)`.
2. The method runs this parameterised query (`apps/api/src/inventory/inventory.service.ts:101–115`):

```
SELECT
  id,
  product_variant_id AS "productVariantId",
  current_quantity   AS "currentQuantity"
FROM product_batch
WHERE product_variant_id = ${variantId}::uuid
  AND expiry_date >= CURRENT_DATE
  AND status <> 'DEPLETED'
  AND current_quantity > 0
ORDER BY expiry_date ASC, batch_number ASC
FOR UPDATE
```

3. PostgreSQL locks every matching row. A second checkout for the same variant waits at this point until the first transaction commits or rolls back.
4. The method walks the locked rows, earliest expiry first. It takes `min(currentQuantity, remaining)` from each row. A batch that reaches zero becomes `DEPLETED`.
5. If the rows cannot cover the request, it throws `OutOfStockError` (HTTP 409) with the requested and available quantities. The whole transaction rolls back, so no batch changes.
6. Otherwise it returns one `DecrementResult` (`batchId`, `qtyTaken`, `depleted`) per batch. Checkout writes one `order_item` per result.
7. The commit releases the locks. The CHECK `current_quantity >= 0` is a last safety net.

Restock (`restockOrderItems`) uses `increment` in SQL, never restocks an expired batch, and turns a `DEPLETED` batch back to `ACTIVE`.



6.3 Two-step checkout with weight-based quote and payment webhook

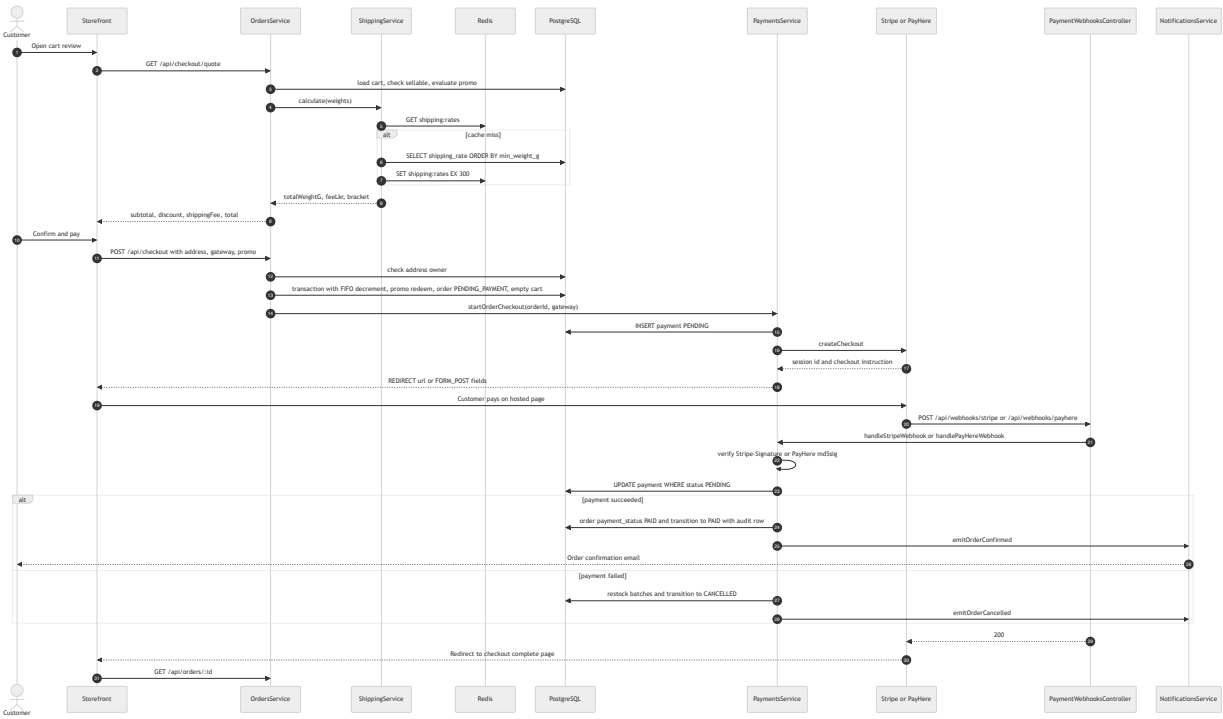
Walkthrough

Step 1: quote (cart review).

1. `GET /api/checkout/quote?promoCode=` reaches `OrdersController.quote`, then `OrdersService.quote`.
2. The service loads the cart and checks that every variant, product and seller is still sellable (`assertSellable`).
3. `PromoCodesService.evaluate` computes the discount.
4. `ShippingService.calculate` sums `unitWeightG × quantity` (a bundle uses `total_weight_g`). It loads the brackets from Redis key `shipping:rates`, or from the `shipping_rate` table on a miss (then caches them for 300 s). It picks the bracket where `min_weight_g <= total <= max_weight_g`. No bracket gives 404.
5. The response holds `subtotal`, `discount`, `shippingFee`, `bracket`, `total` and `totalWeightG`, so the customer sees the shipping fee before paying.

Step 2: checkout and payment.

6. `POST /api/checkout` with `shippingAddressId`, `gateway` (default STRIPE) and `promoCode`. The service checks that the address belongs to the customer (403 otherwise) and that the cart is not empty.
7. One transaction: FIFO decrement (§6.2), promo re-evaluated on the committed subtotal and redeemed with a conditional `UPDATE ... used_count + 1`, order created as `PENDING_PAYMENT` with the snapshots, cart emptied.
8. After the commit, `PaymentsService.startOrderCheckout` creates a `PENDING` payment and asks the gateway for a hosted checkout. Stripe returns a Checkout Session URL (`REDIRECT`). PayHere returns a signed form (`FORM_POST`).
9. The customer pays on the gateway page. The storefront return page polls `GET /api/orders/:id`. The redirect alone never marks an order as paid (ADR-009).
10. The gateway calls the webhook. `/api/webhooks/*` is `@Public`, exempt from CSRF and not rate limited. Signatures:
    - Stripe: `verifyStripeWebhook` calls `Stripe.webhooks.constructEvent` on the raw body with the `Stripe-Signature` header and a 300 s tolerance (`STRIPE_WEBHOOK_TOLERANCE_SEC`).
    - PayHere: `verifyPayHereNotification` checks the merchant id and recomputes `md5sig = MD5(merchant_id + order_id + amount + currency + status_code + MD5(secret))`, compared with `timingSafeEqual`.
11. `applyPaymentEvent` finds the payment. If it is no longer `PENDING`, the event is a duplicate. It checks the amount, then flips the status with a conditional `updateMany ... WHERE status = PENDING`, so a repeated event cannot apply twice.
12. Paid: `onOrderPaid` sets `payment_status = PAID` and calls `OrderStatusService.transition(PAID)` in one transaction, then emits `order.confirmed` (email).
13. Failed: `onOrderPaymentFailed` restocks, cancels the order and emits `order.cancelled`.
14. Paid after the order was already cancelled (for example by the stale-order job): `refundLatePayment` refunds the money.



6.4 Daily Subscribe & Save billing

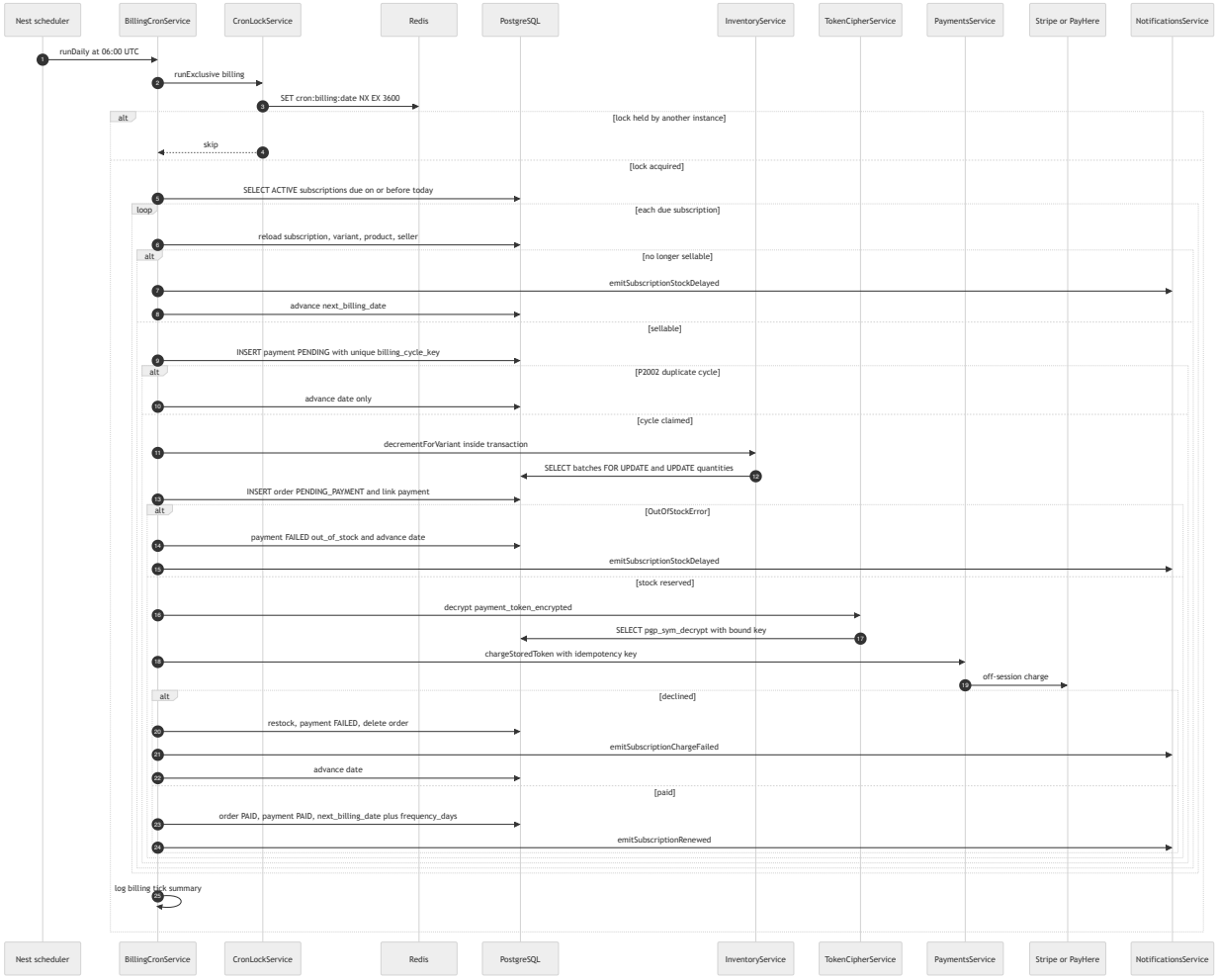
Walkthrough

1. `BillingCronService.runDaily` has `@Cron(CronExpression.EVERY_DAY_AT_6AM)` (`0 6 * * *`). `main.ts` sets `process.env.TZ = 'UTC'`, so it runs at 06:00 UTC, 11:30 in Sri Lanka (ADR-008).
2. `CronLockService.runExclusive('billing', ...)` runs `SET cron:billing:<yyyy-mm-dd> <instanceId> NX EX 3600`. Only the instance that wins runs the job. The lock is not released, so the job cannot run twice on the same day. If Redis is down, the job is skipped.
3. `runBilling` selects `ACTIVE` subscriptions with `next_billing_date` on or before today (index `(status, next_billing_date)`). Each one runs in `processOne`. An error in one subscription is logged and counted. It does not stop the others.
4. `processOne` re-reads the subscription. If the variant, product or seller is no longer sellable, it emits `subscription.stockDelayed` and advances the date.
5. It computes the price: `unit price × quantity × (1 − discount_rate)` plus the shipping fee from `ShippingService.calculate`.
6. **Claim the cycle.** It inserts a `PENDING` payment with `billing_cycle_key = <subscriptionId>:<yyyy-mm-dd>`. The key is unique, so a second run hits `P2002`, advances the date and returns `skipped`. No double charge is possible.
7. **Reserve stock.** In one transaction: `decrementForVariant` (FOR UPDATE, §6.2) and an order in `PENDING_PAYMENT`, linked to the payment. Short stock gives `OutOfStockError`: the payment becomes `FAILED` with `out_of_stock`, `subscription.stockDelayed` is emitted, and the date advances. Nothing is charged.
8. **Charge.** `TokenCipherService.decrypt` runs `pgp_sym_decrypt`. `PaymentsService.chargeStoredToken` charges the stored token with the cycle key as the gateway idempotency key: a Stripe off-session PaymentIntent or a PayHere merchant charge. A gateway exception is turned into `gateway_error`.

9. **Declined.** In one transaction: restock, payment `FAILED` with the reason and `order_id = null`, order deleted. Then `subscription.chargeFailed` is emitted and the date advances.
10. **Paid.** In one transaction: order and payment `PAID`, and `next_billing_date` moved forward by `frequency_days` with a conditional update on the old date. Then `subscription.renewed` is emitted.
11. The date always advances from the cycle date, not from today, so a late run keeps the schedule. The run logs `due`, `renewed`, `outOfStock`, `chargeFailed`, `skipped` and `errored`.

Retries. The job does **not** retry a declined charge in a later run. A declined cycle is recorded as `FAILED` and the customer is emailed (`subscription.chargeFailed`). The next attempt is the next cycle. The only retry is the ioredis client retrying a single Redis command (`maxRetriesPerRequest: 3`).

Manual trigger. `POST /api/admin/subscriptions/billing/run` (ADMIN) calls `runBilling` directly, without the cron lock. It returns 404 when `NODE_ENV=production` (`subscriptions-admin.controller.ts:52-60`). The unique cycle key still prevents double charges.



6.5 Order status state machine

`OrderStatusService.transition` is the only writer of `order.status`. It reads the current status, checks `LAWFUL_TRANSITIONS`, applies role rules, then runs `updateMany ... WHERE status = <read status>`. If another request changed the order first, the caller gets 409. Every transition writes an audit row (`apps/api/src/orders/order-status.service.ts`).

From	Allowed to	Who
`PENDING_PAYMENT`	`PAID`, `CANCELLED`	Payment webhook; customer, admin, stale-order job (cancel)
`PAID`	`PACKING`, `CANCELLED`	Fulfilment staff; admin cancel with refund
`PACKING`	`SHIPPED`, `CANCELLED`	Fulfilment staff; admin cancel
`SHIPPED`	`DELIVERED`, `CANCELLED`	Rider or staff; admin force-cancel
`DELIVERED`	`REFUNDED`	Admin full refund
`CANCELLED`, `REFUNDED`	none (terminal)	

Customers may only cancel their own `PENDING_PAYMENT` orders. Riders may only move `SHIPPED` to `DELIVERED`. Sellers cannot change order status.

---

7. Security architecture

Control	Design	Evidence
Authentication	HS256 JWT, `JWT_SECRET` at least 32 characters or the API refuses to start. Customers 15 days, other actors 8 hours. Payload `sub`, `role`, `actorType`, `jti`.	`auth/auth.service.ts`, `auth/strategies/jwt.strategy.ts`
Token transport	Browsers: `auth_token` cookie, `HttpOnly`, `SameSite=Strict`, `Secure` in production. Mobile: `Authorization: Bearer`, token kept in iOS Keychain or Android Keystore.	`auth/auth.controller.ts:35–50`, `apps/delivery/pubspec.yaml` (`flutter_secure_storage`)
Logout and revocation	Logout writes `jti:<id>` to Redis with TTL equal to the remaining token life. `JwtStrategy.validate` rejects a listed JTI with 401 `Token revoked`. It also re-checks account standing (cache 60 s), so a deactivated or re-roled account gets 401.	`redis/redis.service.ts:44–53`, `auth/strategies/jwt.strategy.ts:58–90`
Passwords	bcrypt cost 12 for all actor tables. Login runs a dummy compare for unknown emails. Password rule: 8 to 128 characters, at least one letter and one digit.	`auth/auth.service.ts:20`, `auth/dto/register.dto.ts`
CSRF	Double-submit cookie. `CsrfMiddleware` runs on every route. GET, HEAD and OPTIONS are exempt and issue the `csrf-token` cookie (24 random bytes, `SameSite=Strict`, not HttpOnly). Other verbs need `X-CSRF-Token` equal to the cookie (`timingSafeEqual`), otherwise 403. `/api/webhooks/*` is exempt because it is signature-verified.	`common/middleware/csrf.middleware.ts:28–60`, `app.module.ts` (`forRoutes('*')`)
RBAC	Global guards in order: `JwtAuthGuard` (routes are private unless `@Public()`), `UserThrottlerGuard`, `RolesGuard` (`@Roles(...)` on class or method, 403 `Role X not permitted`). 14 route handlers are `@Public()`.	`app.module.ts` providers, `auth/guards/roles.guard.ts`, `auth/guards/jwt-auth.guard.ts`

Control	Design	Evidence
Ownership checks	Services compare the owner id with the caller: address and order (`OrdersService.checkout`, `getOrder`, `cancelUnpaid`), subscription (`SubscriptionsService.requireOwned`), payment simulate, seller products (`ProductsAdminService.assertOwner`), rider assignment and proof upload, seller product image upload. Covered by `apps/api/test/ownership.e2e-spec.ts`.	`orders/orders.service.ts`, `subscriptions/subscriptions.service.ts:295-302`, `products/products-admin.service.ts:511`, `uploads/uploads.service.ts:121,139`
Payment token encryption	`TokenCipherService.encrypt` runs `SELECT pgp_sym_encrypt(${plaintext}::text, ${key}::text)` and `decrypt` runs `pgp_sym_decrypt(${bytes}::bytea, ${key}::text)`, both as `Prisma.sql` tagged templates, so the key is a bound parameter and not part of the query text. Key: `SUBSCRIPTION_TOKEN_ENC_KEY`, at least 16 characters, checked at start-up. No `cipher-algo` option is passed, so pgcrypto uses its default cipher (AES-128). The column is `bytea` and no endpoint returns it. Terraform stores the key as an SSM SecureString encrypted with a KMS key.	`subscriptions/token-cipher.service.ts`, `infra/terraform/ssm.tf`
Card data	Never stored. Only gateway references (`gateway_payment_id`) and encrypted saved-card tokens.	ADR-009, `schema.prisma` model `Payment`
Webhook integrity	Stripe HMAC-SHA256 with a 300 s replay window. PayHere MD5 `md5sig` with constant-time compare. Amount checked against the payment row. Conditional status update makes delivery idempotent.	`payments/gateways/stripe.gateway.ts:34-49`, `payhere.gateway.ts:64-87`, `payments.service.ts:254-296`
Rate limiting	Two throttlers stored in Redis. `default`: 100 requests per minute per user (per IP when signed out), one budget across all routes. `auth`: 20 per 15 minutes per IP, only on `register`, `login` and `sellers/apply`. Webhooks and health are exempt. 429 with `Retry-After`. Limits can be overridden by `THROTTLE_*` variables for load tests.	`common/guards/throttler.config.ts`, `common/guards/user-throttler.guard.ts`
HTTP headers (API)	helmet: CSP `default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'none'`, HSTS 1 year with sub-domains, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`.	`config/http-app.ts:49-63`
HTTP headers (storefront)	CSP built in `next.config.mjs`: only self, the API origin, the media origin, Stripe and PayHere.	`apps/storefront/next.config.mjs`
CORS	Explicit origin list from `CORS_ORIGINS` (default ports 3000, 3001, 3002), `credentials: true`, allowed headers `content-type`, `x-csrf-token`, `authorization`, `accept-language`.	`config/http-app.ts:35-45`
Input validation	Global `ValidationPipe` with `whitelist`, `forbidNonWhitelisted`, `transform`. Errors return 422. UUID params checked with `ParseUUIDPipe`.	`config/http-app.ts:77-84`

Control	Design	Evidence
SQL injection	Prisma query builder everywhere. The five raw statements (FIFO lock, promo redeem, two pgcrypto calls, health `SELECT 1`) all use tagged templates with bound parameters. No `$queryRawUnsafe` or `$executeRawUnsafe` exists in `apps/api/src`.	`inventory/inventory.service.ts:101`, `promo-codes/promo-codes.service.ts:74`, `token-cipher.service.ts:31, 45`
File uploads	Presigned PUT URLs valid 300 s, UUID object keys. Before use, the object is checked: owner, size at most 5 MB, magic bytes JPEG, PNG, WebP or HEIC. Failing objects are deleted.	ADR-014, `uploads/uploads.service.ts`, `uploads/dto/presign.dto.ts:6`
Secrets and configuration	`validateEnv` refuses to start in production if a secret is missing or still a development value. `PAYMENTS_MODE=live` requires real Stripe and PayHere keys.	`config/env.validation.ts`, `payments/payments.config.ts`
Audit	Privileged actions and every order transition write `audit_log` rows (actor type, actor id, action, target, metadata).	`audit/audit.service.ts`, `orders/order-status.service.ts`
Mobile transport	Certificate pinning by SHA-256 fingerprint (`API_CERT_SHA256`). Release builds refuse to start without pins.	`apps/delivery/lib/app/certificate_pinning.dart`
TLS	Terraform ALB uses `ELBSecurityPolicy-TLS13-1-2-2021-06` (TLS 1.3 and 1.2). CloudFront minimum is `TLSv1.2_2021`. So the design allows TLS 1.2 and is not "TLS 1.3 only". On Render and Vercel, TLS is provided by the platforms. Their TLS settings were not measured.	`infra/terraform/alb.tf:48`, `infra/terraform/storage.tf:72`

---

8. Scheduled jobs

All jobs use `@nestjs/schedule`. Times are UTC because `main.ts` sets `process.env.TZ = 'UTC'` before Nest starts (ADR-008). Every job takes a Redis lock first through `CronLockService.runExclusive`, so only one API instance runs it.

Job	Class and method	Cron expression	Lock key and TTL	What it does	Notifications
Subscribe & Save billing	`BillingCronService.runDaily`	`0 6 * * *` (`EVERY_DAY_AT_6AM`), 06:00 UTC	`cron:billing:<date>`, 3600 s	Bills due subscriptions (§6.4)	`subscription.renewed`, `.stockDelayed`, `.chargeFailed`

Job	Class and method	Cron expression	Lock key and TTL	What it does	Notifications
Batch expiry sweep	`ExpiryCronService.runDaily`	`0 6 * * *` (`EVERY_DAY_AT_6AM`), 06:00 UTC	`cron:expiry-sweep:<date>`, 3600 s	Marks batches with expiry before today as `EXPIRED`. Marks `ACTIVE` batches expiring within 7 days as `EXPIRING_SOON`. Re-running the same day moves no rows.	`batch.expired`, `batch.expiringSoon` (admins and owning seller)
Stale unpaid orders	`StaleOrderCronService.runScheduled`	`*/5 * * * *` (every 5 minutes)	`cron:stale-orders:<HHMM>:<date>`, 240 s	Cancels up to 200 `PENDING_PAYMENT` orders older than `STALE_ORDER_TIMEOUT_MINUTES` (default 30), restocks unexpired batches, cancels the pending checkout	`order.cancelled`

Files: `apps/api/src/subscriptions/billing-cron.service.ts:69`, `apps/api/src/batches/expiry-cron.service.ts:52`, `apps/api/src/orders/stale-order-cron.service.ts:32`.

Design note. SDS v01 §7.5 asked for a separate scheduled task with its own database account. The jobs run inside the API tasks and share the API's credentials. This is a recorded deviation in `infra/terraform/README.md`.

---

9. API surface, error handling and internationalisation

9.1 API surface

- Base path `/api` (global prefix). `/api/v1/*` is accepted as an alias (`apiVersionAlias` in `common/middleware/http-request.middleware.ts`).
- **116 route handlers in 30 controllers.** `docs/api-reference.md` lists every route with method, path and allowed roles.
- Money is returned as decimal strings in LKR. Order numbers are shown as `BB-000123`.

Module	Routes	Base paths	Roles
auth	5	`/auth/register`, `/login`, `/logout`, `/me`, `/csrf`	Public (register, login, csrf), any signed-in
addresses	3	`/addresses`	CUSTOMER
products	12	`/products`, `/admin/products`	Public; ADMIN, SELLER
bundles	7	`/bundles`, `/admin/bundles`	Public; ADMIN, SELLER
batches	2	`/admin/products/:productId/batches`, `.../variants/:variantId/batches`	ADMIN, SELLER
cart	4	`/cart`, `/cart/items`	CUSTOMER
orders	8	`/checkout/quote`, `/checkout`, `/orders`, `/admin/orders`	CUSTOMER; ADMIN, FULFILMENT_STAFF

Module	Routes	Base paths	Roles
payments	5	`/webhooks/stripe`, `/webhooks/payhere`, `/payments/setup`, `/payments/:id/simulate`	Public (webhooks); CUSTOMER
subscriptions	11	`/subscriptions`, `/admin/subscriptions`	CUSTOMER; ADMIN
fulfilment	5	`/admin/fulfilment`	ADMIN, FULFILMENT_STAFF
delivery	10	`/delivery/*`, `/admin/orders/:id/assign`, `/admin/delivery-assignments`, `/admin/riders`	DELIVERY; ADMIN, FULFILMENT_STAFF
refunds	5	`/orders/:id/refund-requests`, `/account/refund-requests`, `/admin/refund-requests`	CUSTOMER; ADMIN
reviews	6	`/products/:productId/reviews`, `/reviews`, `/admin/reviews`	Public; CUSTOMER; ADMIN
sellers	10	`/sellers/apply`, `/seller/*`, `/admin/sellers`	Public (apply); SELLER; ADMIN
shipping	4	`/admin/shipping-rates`	ADMIN
promo-codes	3	`/admin/promo-codes`	ADMIN
analytics	3	`/admin/analytics/*`	ADMIN
staff	3	`/admin/staff`	ADMIN
support	6	`/support/tickets`, `/admin/support/tickets`	CUSTOMER; ADMIN
uploads	2	`/uploads/proof-photo`, `/uploads/product-image`	DELIVERY; ADMIN, SELLER
health	2	`/health`, `/health/ready`	Public
Total	**116**		

Main flows (from `docs/api-reference.md`):

Flow	Calls
Checkout	`GET /api/checkout/quote` → `POST /api/checkout` → hosted page → webhook (live) or `POST /api/payments/:paymentId/simulate` (mock) → `GET /api/orders/:id`
Subscribe & Save	`POST /api/payments/setup` → card saved at gateway → `POST /api/payments/setup/complete` → `POST /api/subscriptions`
Refund	`POST /api/orders/:id/refund-requests` → `POST /api/admin/refund-requests/:id/approve` (optional partial amount and restock)
Delivery	`POST /api/admin/orders/:id/assign` → `POST /api/delivery/assignments/:id/status` → `POST /api/uploads/proof-photo` → `POST /api/delivery/assignments/:id/proof`
Product image	`POST /api/uploads/product-image` → browser `PUT` to presigned URL → `POST /api/admin/products/:id/images`

9.2 Error handling

Case	Response	Source
DTO validation failure, unknown field	422 with per-field messages	`ValidationPipe` in `config/http-app.ts`
Missing or bad token	401 (`Malformed token`, `Token revoked`, `Account is no longer active`)	`auth/strategies/jwt.strategy.ts`
Wrong role, CSRF mismatch, not the owner	403	`RolesGuard`, `CsrfMiddleware`, service ownership checks
Unknown record	404	services (`NotFoundException`)
Out of stock, concurrent status change, promo used up	409	`OutOfStockError`, `OrderStatusService.transition`, `PromoCodesService.redeem`
Illegal order transition	400 `Illegal transition`	`OrderStatusService.assertTransition`
Too many requests	429 with `Retry-After`	throttler guards
Bad webhook signature	400 `Webhook signature mismatch`	gateway verify functions
Dependency down at readiness	503 from `/api/health/ready`	`health/health.controller.ts`
Invalid production configuration	Process exits with code 1 at start-up	`config/env.validation.ts`, `main.ts`

Body shape: `{ statusCode, message, error }` (NestJS default). Email and push failures are logged and never fail the request (`NotificationsService.on`, `MailerService.send`). EventEmitter2 is in-process with no retry, so a crash between commit and send loses that email (comment in `notifications.service.ts`).

9.3 Internationalisation

Part	Design	Status
Storefront	next-intl, locales `en` (default) and `si`, `localePrefix: 'never'`, locale in the `NEXT_LOCALE` cookie (ADR-010). Messages in `apps/storefront/i18n/messages/en.json`, `si.json`.	Implemented
API client	The storefront sends `Accept-Language` with each API call.	Implemented (`apps/storefront/lib/api.ts:76-78`)

Part	Design	Status
API	`I18nService` with `en.json` and `si.json` catalogues. `I18nExceptionFilter` translates exception messages from `Accept-Language`.	**Not active at runtime.** The filter is defined in `common/filters/i18n-exception.filter.ts` but is not registered (no `useGlobalFilters` or `APP_FILTER` in `main.ts`, `config/http-app.ts` or `app.module.ts`). API errors are returned in English. `docs/api-reference.md` says messages are translated; that is not true until the filter is registered. The filter also reads only `Accept-Language`, not the `NEXT_LOCALE` cookie.
Admin, seller	English only (no next-intl dependency).	By design

---

10. Deployment and CI/CD

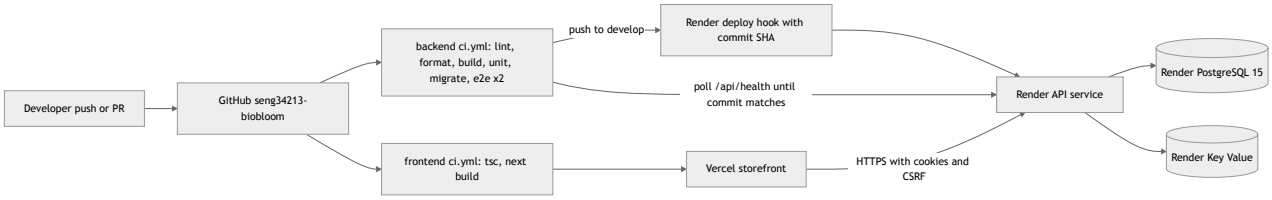
10.1 Environments

Item	Local development	Staging (as deployed)	Production design (`infra/terraform`)
API	Docker Compose `api` on port 4000	Render web service, `https://biobloom-api.onrender.com`	ECS Fargate, 1 to 3 tasks, scale at 70% CPU, behind an ALB
Database	`postgres:15-alpine` (host port 5433)	Render PostgreSQL 15	RDS PostgreSQL 15 `db.t3.small`, 7-day backups, UTC
Redis	`redis:7-alpine`	Render Key Value	ElastiCache Redis 7.1 `cache.t3.micro`, TLS and auth token
Storefront	Compose port 3000	Vercel, `https://biobloom-storefront.vercel.app`	Vercel (SDS v01 §8.1, not in Terraform)
Admin, seller	Compose ports 3001, 3002	No staging URL recorded	Not in Terraform
Media	MinIO (ports 9000, 9001)	Not recorded	Private S3 bucket behind CloudFront
Email	MailHog (1025, UI 8025)	Not recorded	SendGrid SMTP
Secrets	Development values in `docker-compose.yml`	Platform environment variables (not inspected in this review)	SSM SecureStrings encrypted with KMS, injected as task secrets
Deploy trigger	Manual	Push to `develop` in the team repository	`develop` to staging, `main` to production after approval

10.2 Staging as deployed (Render, Vercel, GitHub Actions)

Staging runs from the team polyrepo `seng34213-biobloom` (`backend`, `frontend`, `mobile`, `documents`), not from this monorepo.

- **Backend** (`backend/.github/workflows/ci.yml`). Job `build-and-test` on every push and PR to `main` and `develop`: Postgres 15, Redis 7 and MailHog service containers; `npm ci`, lint, format check, build, unit tests, `prisma:deploy`, end-to-end tests run twice to prove repeatability. Job `deploy-staging` runs only on a push to `develop`: it posts the Render deploy hook with the commit SHA, then polls `GET /api/health` every 20 s (up to 45 times, 15 minutes) until the `commit` field equals the pushed SHA. Render keeps the old version live if the new one fails its health check.
- **Frontend** (`frontend/.github/workflows/ci.yml`). Type check and build on Node 20. Vercel builds and hosts the storefront.



Important difference. The team backend repository has **19 Prisma models** (`backend/prisma/schema.prisma`). This monorepo, which this document describes, has **25**. So staging does not yet run the full design (for example, no `ProductImage`, `Bundle`, `BundleItem`, `PromoCode`, `Payout`, `FavoriteProduct` models in the team schema). The team frontend uses `next` 14.2.33; the monorepo apps use 14.2.5.

10.3 Monorepo pipelines (`.github/workflows/`)

`ci.yml` implements SDS v01 §8.2 stages 1 to 6:

Job	What it runs
`lint`	`pnpm install --frozen-lockfile`, Prisma generate, `pnpm lint`, `pnpm typecheck`, `pnpm format:check`, actionlint
`unit`	API Jest with coverage gate (`test:cov`), Vitest for `ui`, `storefront`, `admin`, `seller`
`integration`	Postgres 15, Redis 7, MailHog; `migrate:deploy`, seed, Supertest suites (`test:e2e`, 11 suites in `apps/api/test`)
`audit`	`pnpm audit --audit-level=high`
`flutter`	`flutter pub get`, `flutter analyze`, `flutter test`
`build`	hadolint and Docker build of `api`, `storefront`, `admin`, `seller`
`push`	API and migrator images to ECR, only on `develop` and only when `vars.AWS_ROLE_ARN` is set

`deploy.yml` implements stages 7 and 8 for AWS: runs after CI succeeds; runs migrations as a one-off ECS task; registers a new task definition; rolling update with 100% minimum healthy and 200% maximum plus the deployment circuit breaker; health check 12 × 10 s (120 s); rolls back to the previous task definition on failure. `main` deploys to the `production` environment, which needs a manual approval.

These two workflows target AWS. Staging does not use them (§10.2).

10.4 Designed AWS infrastructure (`infra/terraform/`)

File	Creates
`network.tf`	VPC, two public and two private subnets, NAT gateway, security groups
`alb.tf`	HTTPS load balancer (TLS 1.3/1.2 policy), HTTP redirect, `/api/health` checks
`ecs.tf`	Fargate cluster, API service (1 to 3 tasks, 70% CPU target), migrate task
`ecr.tf`	API image repository with lifecycle policy
`rds.tf`	PostgreSQL 15 `db.t3.small`, 7-day backups, `timezone = UTC`
`elasticache.tf`	Redis 7.1 with TLS and an auth token
`storage.tf`	Private media bucket with SSE, CloudFront with origin access control, CORS
`ssm.tf`	KMS key, generated secrets (DB, Redis, JWT, token key), external secret slots
`iam.tf`	Task roles, GitHub OIDC roles for ECR push and deploy
`monitoring.tf`	SNS topic, alarms for 5xx, p95 latency, unhealthy hosts, RDS CPU and storage

`infra/load-test/k6/` holds k6 scripts `checkout.js` and `subscription-cron.js`.

10.5 Quality gates: designed versus measured (25 October 2026, monorepo)

Gate	Design target	Measured	Met?
API unit tests	All pass	24 suites, 231 tests, all pass	Yes
API coverage (Jest gate `apps/api/jest.config.json`)	65% statements, 50% branches, 65% functions, 65% lines	31.24% statements, 23.38% branches, 21.82% functions, 31.54% lines	**No**
Coverage target in Definition of Done (`docs/development-issues.md`)	80%	31.24% statements	**No**
Frontend unit tests (Vitest)	All pass	storefront 11, admin 7, seller 2, `packages/ui` 20, all pass	Yes
Dependency audit	Zero high or critical (SDS v01 §8.2)	163 advisories: 15 low, 69 moderate, 69 high, 10 critical. Critical packages include `next` (Authorization Bypass in Middleware, fixed in 14.2.25 and later), `handlebars`, `tinypool`, `proxy-addr`, `vitest`	**No**

Because the coverage gate and the audit gate fail, the monorepo `ci.yml` would not pass the `unit` and `audit` jobs today.

---

11. ADR index

ADR	Title	Decision in one line
ADR-001	Single pnpm monorepo, Flutter app alongside	One pnpm workspace for API, three web apps and shared packages; Flutter in `apps/delivery` outside the workspace.
ADR-002	Multi-seller marketplace instead of a single vendor	Sellers apply and are approved; every product belongs to one seller; sellers manage only their own catalogue.
ADR-003	Email-only customer notifications, FCM push for riders	EventEmitter2 events send email for every domain event; riders also get FCM push on `delivery.assigned`.
ADR-004	PostgreSQL 15 with Prisma; pgcrypto from init SQL	Prisma 6 on Postgres 15, UUID keys, Decimal money; pgcrypto created by migration and init SQL; tokens encrypted with `pgp_sym_encrypt`.
ADR-005	Separate identity table per actor type	Four actor tables, one login endpoint, `actorType` in the JWT selects the table.
ADR-006	JWT cookie, Redis JTI blocklist and double-submit CSRF	HttpOnly `SameSite=Strict` cookie or Bearer, JTI revoked in Redis on logout, double-submit CSRF on mutating calls.
ADR-007	FIFO allocation with `SELECT ... FOR UPDATE`	Lock valid batches earliest expiry first inside the order transaction; 409 and rollback when short.
ADR-008	All scheduling and date arithmetic in UTC	`TZ=UTC` before bootstrap; dates stored at UTC midnight; billing at 06:00 UTC.
ADR-009	Hosted checkout with signed webhooks driving order state	Stripe Checkout or PayHere form; only verified webhooks change payment and order state.
ADR-010	Cookie-based locale without URL prefixes	next-intl with `localePrefix: 'never'` and the `NEXT_LOCALE` cookie; `en` and `si`.
ADR-011	Cancel unpaid orders after 30 minutes	A scheduled job cancels `PENDING_PAYMENT` orders older than the timeout and restocks unexpired batches.
ADR-012	Mock and live payment modes	`PAYMENTS_MODE=mock` simulates the hosted page through the real code path; `live` needs real keys; mock refused in production.
ADR-013	Re-open failed delivery assignments with an attempt counter	A failed assignment is re-opened to `ASSIGNED` with `attempts + 1` instead of creating a new row.
ADR-014	Presigned S3 uploads verified by magic bytes	Clients upload with short-lived presigned PUT URLs; the API checks owner, size and magic bytes before use.

12. Known gaps and verification register

These items are where the code differs from a design claim, from SDS v01, or from the project documents. Each is stated with evidence so the team can fix or accept it.

#	Claim	Real status	Evidence
1	Webhooks verified by HMAC-SHA256 (SDS v01 §4.2)	True for Stripe only. PayHere uses its MD5 `md5sig` scheme.	`payments/gateways/payhere.gateway.ts:64–87`

#	Claim	Real status	Evidence
2	Payment tokens encrypted with AES-256 under AWS KMS (SDS v01 §7.3)	`pgp_sym_encrypt` with an application key and no `cipher-algo` option, so pgcrypto's default (AES-128). KMS only protects the stored key in the Terraform design.	`subscriptions/token-cipher.service.ts:31-50`, `infra/terraform/ssm.tf`
3	API error messages translated to Sinhala (`docs/api-reference.md`)	`I18nExceptionFilter` is never registered, so translation does not happen.	`common/filters/i18n-exception.filter.ts`, `app.module.ts`, `config/http-app.ts`
4	ADR-007 names `InventoryService.allocateFifo`	The method is `decrementForVariant`. The query filters `status <> 'DEPLETED'` and `expiry_date >= CURRENT_DATE`, not "status ACTIVE or EXPIRING_SOON" (same effect, because the sweep sets `EXPIRED` only for past dates).	`inventory/inventory.service.ts:89-115`
5	Billing failures are retried	Not implemented. A declined cycle is recorded and emailed; the next attempt is the next cycle.	`subscriptions/billing-cron.service.ts:244-268`
6	Cron jobs under a dedicated, restricted service account (SDS v01 §7.5)	Jobs run in the API process with the API credentials. Recorded deviation.	`infra/terraform/README.md`
7	TLS 1.3 enforced (SDS v01 §7.3)	Terraform allows TLS 1.3 and 1.2 at the ALB and TLS 1.2 or later at CloudFront. Render and Vercel TLS not measured.	`infra/terraform/alb.tf:48`, `storage.tf:72`
8	Zero high or critical vulnerabilities (SDS v01 §8.2)	69 high and 10 critical on 25 October 2026. `pnpm audit` exits with code 1.	`pnpm audit --audit-level=high`
9	65% coverage gate (SDS v01 §8.2), 80% target	31.24% statements. Gate not met.	`apps/api/jest.config.json`, coverage run
10	Staging on AWS ECS (SDS v01 §8)	Staging is on Render and Vercel from the team polyrepo. The AWS design is in Terraform and `deploy.yml`.	§10.2, `infra/terraform/`
11	Staging runs the design in this document	The team backend schema has 19 models; this design has 25.	`backend/prisma/schema.prisma`, `packages/prisma/prisma/schema.prisma`
12	Payouts and favourites	Tables exist; no API, service or UI. Not implemented.	`schema.prisma` models `Payout`, `FavoriteProduct`
13	Manual billing trigger uses the cron lock	It calls `runBilling` directly, without the lock. It is disabled in production and the unique cycle key still blocks double charges.	`subscriptions/subscriptions-admin.controller.ts:52-60`

#	Claim	Real status	Evidence
14	Variant or bundle required on order and cart lines	Enforced only in code; no CHECK constraint. Same for payment owner and non-overlapping shipping brackets.	`migrations/20261022090000_spec_completion/migration.sql`
15	Product names on past orders never change	Names are not snapshotted; order views join the current product.	`orders/orders.service.ts` (`orderDetailInclude`)

End of document.